

Birds of the Same Feather Flock Together: A Dual-Mode Circuit Candidate for Strong PUF-TRNG Functionalities

Kuheli Pratihar¹, Urbi Chatterjee², Manaar Alam¹, Rajat Subhra Chakraborty¹, *Senior Member, IEEE*, and Debdeep Mukhopadhyay¹, *Senior Member, IEEE*

Abstract—Physically Unclonable Functions (PUFs) and True Random Number Generators (TRNGs) are two highly useful hardware primitives to build up the root-of-trust for embedded devices in Internet-of-Things and Cyber-Physical System applications. These applications demand the primitives be lightweight, yet flexible. However, PUFs are designed to offer *repetitive and instance-specific* randomness, whereas TRNGs are expected to be *invariably* random. A challenging but thought-provoking problem from a hardware designer's perspective would be to design a circuit that serves the purpose of both PUF and TRNG depending on the exact requirement of the application. Here, we present a dual-mode PUF-TRNG design that utilises two different hardware-intrinsic properties, i.e., oscillatory metastability of Transition Effect Ring Oscillator (TERO) cell and propagation delay of a buffer within the cell to achieve this goal. A 48.62% reduction in area is accomplished due to the integration in comparison to separate instances of standalone PUFs/TRNG designs, built from Programmable Delay Line (PDL) based Arbiter PUFs (APUFs) and TERO-TRNG. Our final design has a hardware footprint of 618 Look-Up Tables (LUTs) and 447 Flip-Flops (FFs). Furthermore, experimental analysis of the state-of-the-art modelling attacks, reliability attacks on the proposed PUF design shows a prediction accuracy of 55.37% and 50.14% respectively for 5.2M Challenge Response Pairs (CRPs). Additionally, the TRNG passes evaluation through National Institute of Standards and Technology (NIST) Special Publication (SP) 800-22 and German Federal Office for Information Security (BSI) Application Notes and Interpretation of the Scheme (AIS)-31 tests.

Index Terms—True random number generators, physically unclonable functions, transient effect ring oscillator, recurrent neural network, deep learning

1 INTRODUCTION

IDENTIFICATION and Random Number Generation play a crucial role in modern cryptographic systems. There exist two cryptographic primitives that extract randomness from the physical hardware resource they are embedded upon. Physically Unclonable Functions (PUFs), exploit the random physical variations inherent to any manufacturing process to generate device-specific and unclonable fingerprints. On the other hand, True Random Number Generators (TRNGs) produce bitstreams that are independent, unpredictable and uniformly distributed from random physical

phenomena. The circuitry for both PUFs and TRNGs require a very small systematic mismatch such that there exists no bias in their respective responses. While the output for both PUFs and TRNGs are unpredictable, the fundamental difference lies in the fact that for the same challenge, the PUF response is repeatable for every run, whereas for TRNGs, the output varies randomly on every run.

Now, in state-of-the-art literature, mostly four design principles have been followed so far and are as follows:

- *Stand-alone PUF circuits*: The Arbiter PUF (APUF) [1] is the first proposed electrical, integrated Strong PUF. It exploits the intrinsic timing difference between two symmetrically designed paths and generates a single bit response. Using this building block, several other non-linear PUF compositions, e.g.: Interpose PUF [2], LP-PUF [3] etc. have been proposed in the literature. Promising FPGA based weak PUFs include Ring Oscillator (RO) PUF [4], Transition Effect Ring Oscillator (TERO) PUF [5] and the Loop PUF [6]. Additionally, a popular memory based weak PUF construction can be found in [7].
- *Stand-alone TRNG circuits*: Similarly, there have been a large number of TRNG designs that exploit a wide variety of physical phenomena as its noise source. Noise sources like thermal noise and metastability are commonly used entropy source for TRNG designs [8], [9]. Several other well-known TRNG

- Kuheli Pratihar, Manaar Alam, Rajat Subhra Chakraborty, and Debdeep Mukhopadhyay are with the Department of Computer Science and Engineering, Indian Institute of Technology, Kharagpur, West Bengal 721302, India. E-mail: {its.kuheli96, alam.manaar, rschakraborty, debdeep.mukhopadhyay}@gmail.com.
- Urbi Chatterjee is with the Department of Computer Science and Engineering, Indian Institute of Technology, Kanpur, Uttar Pradesh 208016, India. E-mail: urbi.ism@gmail.com.

Manuscript received 26 December 2021; revised 5 October 2022; accepted 20 October 2022. Date of publication 2 November 2022; date of current version 10 May 2023.

This work was supported in part by the Department of Science and Technology (DST), Government of India and in part by IHUB NTIHAC Foundation, C3i IIT Kanpur, India under the Project "Development of Secured Hardware and Automotive Systems".

(Corresponding author: Kuheli Pratihar.)

Recommended for acceptance by X. Cheng.

Digital Object Identifier no. 10.1109/TC.2022.3218986

designs include PLL based TRNGs [10], Multiple Ring Oscillator (MURO) TRNG [11] and the Self-Timed RO TRNG [12].

- *Integrated PUF-TRNG circuit using same entropy source:* Stand-alone implementation of PUFs and TRNGs incur increased hardware footprint. As a solution, designers integrate PUFs and TRNGs into a single building block [13]. In [14], a hardware block called Universal TERO (UTERO) for simultaneous generation of PUF responses and random bits was proposed. A recent work [15] describes a unified approach for generating PUF responses and TRNG bits from a common entropy source.
- *Integrated PUF-TRNG circuit using different entropy sources:* A unified weak PUF and TRNG design utilizing current-steering digital-to-analog converter (DAC) and ring voltage-controlled oscillator (VCO) has been proposed in [16].

However, the trend in security primitive design has been to use different independent circuits for PUFs and TRNGs as the balance between manufacturing process variations for PUFs and randomness from a noise source for a TRNG is challenging to achieve for an integrated design [17]. Furthermore, the above-mentioned designs on integrated PUF-TRNG are a blend of weak PUFs and TRNG requiring large hardware footprint. This provides us a solid motivation to present a unified PUF-TRNG design that brings together a delay-based candidate strong PUF and an oscillatory metastability based TRNG by sharing and reusing a significant amount of the utilized hardware resources suitable for IoT applications. To the best of our knowledge, this is the first work that integrates a *delay based Candidate Strong PUF[†] with an oscillatory metastability based TRNG*.

The evolution of Strong PUF constructions has been significantly influenced by the rapid increase of Machine Learning (ML) based model building attacks. APUFs form the core building block for state-of-the-art Strong PUFs [1], [3], [19]. However, they can be modelled by a linear function using collected challenge-response pairs (CRPs) to build a mathematical replica of the PUF construction. Logistic Regression (LR) in the context of attacking Strong PUFs was introduced in [18], [20]. Another genre of attacks utilizing information about the reliability of PUF responses was presented in [21]. Side channel attacks [22] also aid in ML modelling by extracting intermediate values of the PUF circuit. Research based on learning theory and examination of the model building Probably Approximately Correct (PAC) framework [23] has also been investigated in the literature. Hence, we perform an exhaustive machine learning (ML) as well as deep learning (DL) vulnerability analysis, reliability analysis of our proposed architecture and also *present an architectural tweak using recurrence* to make it resistant against the ML models. The proposed TRNG design satisfies relevant security features as well as passes all the statistical tests whereas the PUF circuit exhibits desirable uniqueness, uniformity and reliability properties along with substantial robustness against state-of-the-art model building attacks.

On the other hand, generating random numbers as well as deriving PUF responses that satisfy the required properties over a wide range of operating conditions is a challenge in itself. Although, dynamic variation of environmental factors on the circuit performance metrics will require extensive experimentation, we have performed a preliminary temperature variation analysis of the proposed PUF design and observed that a large proportion of the CRPs under test return a reliable PUF response w.r.t the response obtained under ambient conditions.

1.1 Motivation

Both PUFs and TRNGs find a wide range of applications in almost all security solutions. For example, *Slender PUF* based authentication scheme [24] takes the advantage of TRNGs to randomly select the sub-string of the response of a PUF instance and sends it for verification along with a part of challenge randomly produced inside the chip.

Recently, a threshold implementation on Strong PUF design [25] has been proposed to provide a provably-secure masking countermeasure against power side channel attacks. As the actual behaviour of the PUF instance is not known prior to the manufacturing of the device, the proposed scheme ensured to operate on any Boolean function in a masked way. It was assumed that the attacker can have access to the netlist of the circuit and the input-output layers and can query the PUF arbitrary number of times. But it cannot directly probe the intermediate values and get access to the random numbers generated inside the chip (refer to Sec.I. B. of [25]). This assumption is essential for any secret sharing scheme. Now, for every run of the PUF characterisation, the secret masks are randomly generated inside the target FPGA by means of a couple of randomly-seeded 31-bit LFSRs. Now as the LFSRs are themselves susceptible to power side channel attacks [26], using the true random number source for the secret share generation can be a better design decision. These scenarios as well as the challenge of balancing the randomness of a noise source versus manufacturing process variations in the construction of integrated PUF-TRNG [17] motivated us to look for effective methodologies that re-configures a TRNG to operate as a PUF like circuit with strong ML robustness. Such a primitive would enable a wide variety of applications and features in the IoT domain without needing additional resources.

1.2 Our Contribution and Paper Organization

The novel contributions of this paper are as follows:

- We design and implement a dual-mode TERO cell (referred to as Design-1) that exhibits the functionalities of both the PUF and the TRNG. We then introduce an auxiliary circuit to extend the TERO cell into a complete hardware structure as Design-2 that can on-the-fly switch to operate as an oscillatory metastability based TRNG and a delay based candidate Strong PUF design. To the best of our knowledge, it is the first compact and unified architecture that effectively blends a delay based candidate Strong PUF and the well-studied TERO TRNG by leveraging two different hardware intrinsic properties in the same circuitry.

[†] The term *Candidate Strong PUF* exists in literature [18]. It indicates PUF constructions that are difficult to model or exhibit strong resistance to modelling attacks.

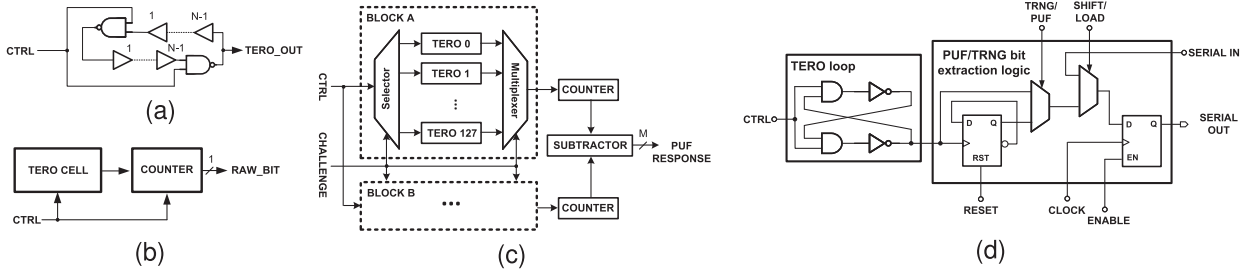


Fig. 1. (a) Generic structure of a TERO cell. (b) Block diagram of the TERO TRNG core. (c) Hardware/software co-design of the TERO-PUF on FPGA. (d) UTERO element generating simultaneous PUF and TRNG bits [14].

- To test the quality of the TRNG, in addition to statistical tests such as the NIST SP800-22 [27] on the random bitstream, we also measure the auto-correlation function (ACF) and verify the Wide Sense Stationarity (WSS) property for the raw bits. Min-entropy estimation on the TRNG output bits has also been done. Furthermore, for verification purposes, sequences of internal numbers were generated and tested using T0-T8 tests in AIS-31 [28]. The random bit sequences generated from the proposed TRNG design passes all the above-mentioned tests.
- We also analyse the robustness to ML based modelling attacks of the co-residing PUF-TRNG architecture. Our initial investigation shows that a linear delay model of the proposed candidate Strong PUF can be formed. Hence, to impede the ML attacks, we present two modified architectures (Design-3A and Design-3B) using challenge obfuscation technique inspired by Recurrent Neural Networks (RNNs), which significantly reduce the prediction accuracy of ML attacks without adversely affecting the reliability of PUF instance. We have used Logistic regression with RProp, Linear Support Vector Machine (SVM), SVM with polynomial kernel, SVM with RBF kernel, Random Forest, Deep Neural Networks and the PyPUF [29] library to check the modeling accuracy for the PUF design. Additionally, we also perform reliability and chosen challenge based attacks on the co-residing PUF-TRNG architecture with the challenge obfuscation mechanism.
- Finally, we have implemented our design with recurrence on the Xilinx Artix-7 FPGA in Nexys-4 DDR board and evaluated the quality metrics of both the PUF and TRNG design. The hardware overhead of the unified PUF-TRNG design with dual-mode operability (Design-2) incurs 152 look-up tables and 16 flip-flops respectively. This integration itself reduces the hardware footprint by 48.62% when compared to separate implementations of PDL based APUF and TERO-TRNG. Additionally, the final design with recurrence (Design-3B) utilizes 618 look-up tables and 447 flip-flops. We prove analytically and validate empirically that our Design-3B has strong resistance against modelling and reliability based attacks achieving an accuracy of 55.37%, 50.14% respectively. Also, we have performed a preliminary analysis of the proposed PUF design by varying the temperature from -25°C to $+70^{\circ}\text{C}$ and

observe that atleast 92% of the CRPs under test return a reliable PUF response w.r.t the response obtained under ambient conditions.

The remainder of the paper is organized as follows. Section 2 provides the necessary background. In Section 3, we discuss the proposed architecture in details. Next, we present the machine-learning analysis of the PUF design and propose a modified architecture to increase the model-building resistance against several state-of-the-art ML algorithms in Section 4. The experimental setup and results are illustrated in Section 5. Section 6 presents some interesting perspectives and lastly, we conclude the paper in Section 7.

2 BACKGROUND

In this section, we start with a general understanding of the TERO cell, the working principles of TERO-TRNG, TERO-PUF and a new hardware primitive UTERO for combined PUF-TRNG functionality. Then, we briefly discuss the delay element and preliminaries of delay-based PUF designs on FPGA. Lastly, we discuss the commonly used metrics for the evaluation of PUFs and TRNGs.

2.1 The TERO Cell and UTERO

The TERO cell is a metastable structure first proposed in [30]. Fig. 1a shows the generic structure of a TERO cell. It comprises of two identical and symmetrical branches with each branch having an initialization stage and some buffers. On initialization with the rising edge of the CTRL signal, two events begin to propagate inside the TERO loop and starts oscillating. Due to CMOS process variations and delay mismatch between the two branches of the TERO cell, these two events continue to propagate until they collide and stop the oscillating state. This behaviour results in a finite number of oscillations resulting in the TERO cell output. Originally proposed for its use as a TRNG (shown in

Fig. 1b), the TERO cell was later found to have the characteristics for its use as a PUF. Fig. 1c shows the TERO-PUF architecture as presented in [5]. Depending on the challenge sent to the device, only two TERO cells from each block oscillate, and the recorded number of temporary oscillations is sent to the software. At the software end, from the difference between the number of oscillations of the chosen TERO cells, bits are selected to obtain the PUF response. A new hardware building block called the UTERO [14] was proposed for independent generation of PUF and TRNG bitstreams. It consists of a TERO loop and a bit extraction logic

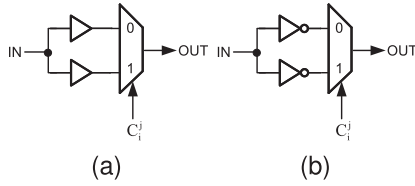


Fig. 2. Basic delay element with (a) buffers, and (b) inverters.

for PUF/TRNG (See Fig. 1d). However, it was reported in [5] that out of all TERO loops implemented, the TERO output fell to a stable logic level '0' in 40.3% of the cases, to level '1' in 30.7% of the cases, and was unstable in 29.0% of the cases. This observation concludes that the final state of the UTERO output cannot be used as an entropy source for generating the PUF response.

2.2 The Delay Element

The Loop PUF [6] is a ring oscillator based PUF and is mainly composed of a chain of N identical controllable delay stages. An individual delay stage of the Loop PUF is realized using a pair of delay elements such as buffers or inverters (See Fig. 2). A bit C_i^j is applied to i^{th} stage via a 2×1 MUX and one of the two elements is selected and then included in the feedback path and gets evaluated sequentially to generate the PUF response.

2.3 FPGA Based APUF Designs

The APUF is mainly composed of two primary components: (a) two parallel structures of cascaded path-swapping switching elements (each path swapping switch is known as a *stage*), and (b) the arbiter itself [1]. Each stage of the APUF consists of a single bit challenge as input and two outputs from the previous stage. The inputs to the first stage are connected to a common enable signal and the outputs of the last stage are connected to the arbiter, which determines which of the signal from the two paths arrived first. Based on this, it generates a single bit known as the PUF response. However, to alleviate the issue of equalizing the delays of the top and bottom paths in a classical APUF due to FPGA routing constraints, an easy-to-design APUF variant known as Programmable Delay Line (PDL) based APUF (PAPUF) was introduced in [31]. In contrast to APUF, PAPUF uses the non-path swapping switches and partially alleviates the problem of design bias in FPGAs. The introduction of additional delay tuning elements help mitigate this design bias. The design approach of the PAPUF makes its implementation better than its classical counterpart.

Evaluation Metrics. The commonly used metrics to evaluate PUFs and TRNGs are:

- 1) **Uniformity:** It estimates the distribution of responses corresponding to a challenge set and is calculated with the Hamming Weight in the responses to 50% (ideal value).
- 2) **Uniqueness:** This property describes the difference in responses obtained when an identical challenge set is given as input to a pair of PUF instances (ideally 50%).
- 3) **Reliability:** A PUF is said to be reliable when the responses are reproducible for an identical challenge

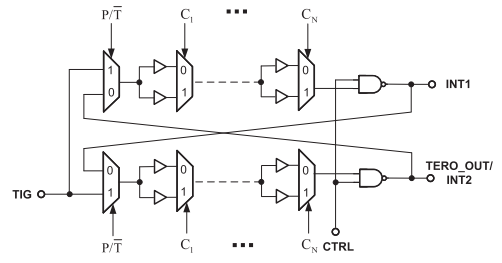


Fig. 3. Proposed architecture of a dual-mode TERO cell.

over time and operating conditions. The ideal value is 100%. However, a maximum error rate of 5% is tolerable and can be corrected using Error-Correcting Codes (ECC) [32].

- 4) **Randomness:** The quality of random numbers play a crucial role in cryptography and therefore a thorough assessment of their properties is necessary. We adapt the NIST SP800-22 Statistical Test Suite and the German Standard AIS-31 for investigating the quality of randomness of the generated bitstream from our TRNG design.

The following section starts with our proposed dual-mode TERO cell and gradually evolves to build a hardware core with a delay-based candidate Strong PUF by reconfiguring a part of this core for its use as TERO TRNG to generate onboard randomness.

3 PROPOSED UNIFIED PUF-TRNG ARCHITECTURE

In this section, we propose the architecture for a unified PUF-TRNG and the design rationale over conventional design criteria. These design criteria include: a compact implementation with little to no additional hardware resources, a dual-mode architecture that works both as a PUF/TRNG on-demand and its feasibility on various platforms.

3.1 Design 1: A Dual-Mode TERO Cell

We first use TERO cell structure as proposed in [33] to develop a unified PUF-TRNG structure. The conventional implementation of TERO-PUF has large hardware footprint as it requires multiple TERO cells to generate the response and is exponentially proportional to the precision of input challenge (refer Fig. 1c). However, the TERO cell used for generating random bitstreams can be re-configured and used as a PUF with minimal hardware overhead. The major crux of our proposed scheme is to develop a primitive by using a *single TERO cell with both PUF and TRNG functionalities*. We achieve this by replacing the buffers of the TERO cell with the delay elements.

As shown in Fig. 3, the TERO cell consists of two symmetrically laid delay chains consisting of a total $2N$ identical challenge-driven delay stages along with one 2×1 multiplexer at the end of every stage. As a particular delay stage is realized using two buffer elements, we can apply challenges to the 2×1 multiplexer at every stage to select one of them for the trigger signal propagation. This enables us to select 2^N combinations of buffers from the single TERO cell using an N -bit binary challenge. This makes our proposed PUF architecture fundamentally distinct from the conventional TERO-PUF design which generates the difference

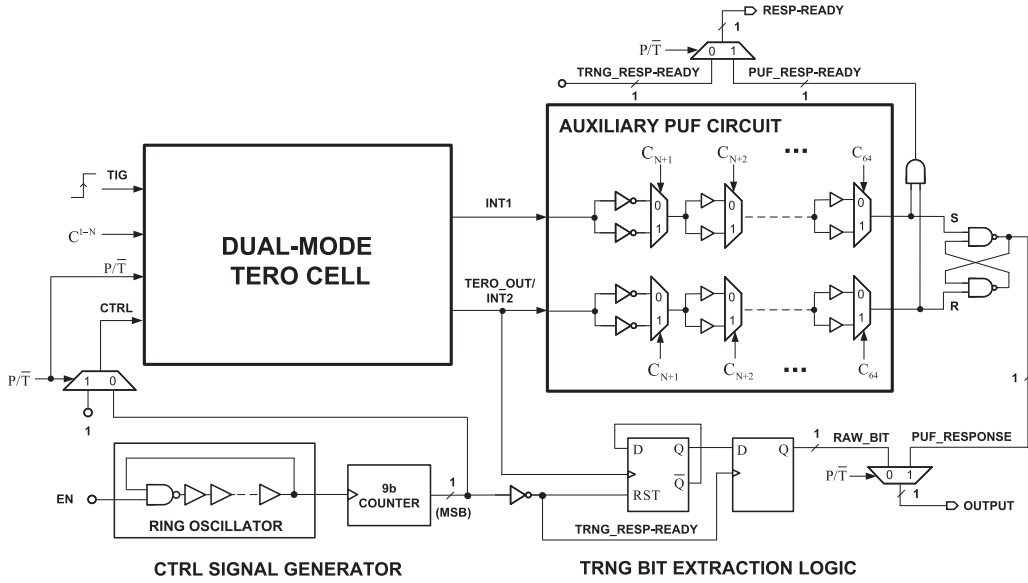


Fig. 4. Unified PUF-TRNG architecture with dual-mode operability.

between the number of temporary oscillations as the PUF response [5]. The natural followup question is: *How can we exploit the delay difference between two chains as its source of device-specific randomness as opposed to the difference in the number of transient oscillations?* The solution we propose is to break the feedback loop in the TERO-cell structure, thereby giving rise to two delay chains as shown in Fig. 3. We achieve this by introducing two additional 2×1 multiplexers before the upper and lower delay chains to control the operability of the cell. Depending on the control signal $P/\bar{T}$, the dual-mode TERO cell either exploits the temporary oscillation of the feedback loop to generate random bits or leverages the delay differences of the two symmetrically laid delay chains to generate the PUF response. To the best of our knowledge, *this is the first design to utilize two very different entropy sources (metastability for TRNG operation and propagation delay for PUF operation) within the same structure*, thereby bringing together the goodness of a candidate Strong PUF and the well-studied TRNG design into a single dual-mode hardware architecture.

But, while designing the proposed scheme, a major challenge we faced was the deterioration of the TRNG quality. Most of the state-of-the-art TERO TRNGs take between 7 – 11 buffers in every cell to generate true random bits depending on the platform on which it is embedded. If we increase the number of buffers to also make it work as a 64 or 128-bit candidate Strong PUF, then the quality of the TRNG may decrease substantially. This issue motivated us to improve the circuit design and customize it to function as a 64-bit Strong PUF and a TRNG with 7-delay element TERO cell. In the next subsections, we use this afore-mentioned dual-mode TERO cell to build a complete circuit structure and proceed with the operating principle of the same.

3.2 Design-2: A Dual-Mode PUF-TRNG Structure

The unified PUF-TRNG circuit mainly consists of four components, a) the dual-mode TERO cell, b) an auxiliary PUF circuit, c) TRNG bit extraction logic, and d) a control signal generator. The overview of the same is shown in Fig. 4. The

core of the TERO-TRNG is the dual-mode TERO cell built using a multiplexer, a chain of N delay elements, and a NAND gate in both the branches. The TERO cell is followed by the TRNG bit extraction block that comprises of a 1-bit counter implemented using a T-flip flop and the output data register. The control signal generator periodically restarts the dual-mode TERO cell during its TRNG operation by using a 9-bit counter driven by a conventional ring oscillator. However, during its operation as a PUF, the control signal is set to high and as a result, the NAND gates in each branch of the TERO loop behave as an inverter. The first N bits of the 64-bit binary challenge are provided identically to the N delay elements of the top and the bottom branch in the dual-mode TERO cell. The remaining $(64 - N)$ challenge bits are provided to the delay elements in the auxiliary PUF circuit. *This design principle eventually eradicates the necessity of including all 64 delay elements in the TERO cell itself, thus, in turn maintaining the good TRNG quality.*

3.3 Operation Principle of Design-2

Initially, we use the $P/\bar{T}$ signal to select the mode of operation for the circuit. This signal is given as an input to the select line of the multiplexers as a result *disabling/enabling the feedback loop*, as the case may be. We discuss the operation in detail as follows:

TRNG Mode. In this case, $P/\bar{T}$ is set to 0, thereby enabling the feedback loop and ensuring the bi-stable operation of the TERO cell. All the challenge bits are set to a specific configuration in this mode thereby selecting a fixed set of delay elements. In the reset phase, the control signal $CTRL = 0$, therefore the output of the TERO loop is 1. When $CTRL = 1$, the TERO cell starts oscillating and continues to do so until it reaches a stable state. The counter is implemented using a T-flip flop and counts the number of temporary oscillations. The output data register gives the least significant bit (LSB) of the total number of temporary oscillations as the internal random bit.

The counter resets at a smaller frequency equal to the RO frequency divided by $2^9 = 512$ for a 9-bit counter. The

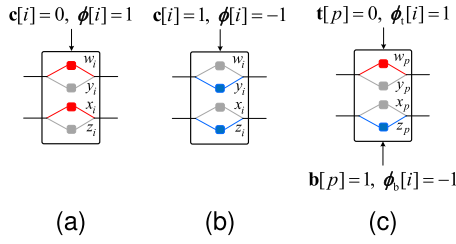


Fig. 5. Depending on the challenge bit $c[i] = \{0, 1\}$, a pair of non-path swapping switches is selected [34] as shown in (a), (b) proposed PUF's i^{th} stage, and (c) p^{th} tuning stage with $t[p]$, $b[p]$.

reason to choose the 9-bit counter will be discussed in detail in Section 5. The duration for which the *CTRL* pulse remains high is the duration for which the TERO loop will oscillate. The TERO loop either reaches the steady state in this interval or keeps on oscillating. The bit extraction logic extracts the LSB from the total number of temporary oscillations when *CTRL* = 1. The *T-Flip Flop* behaves as a 1-bit counter by computing the LSB of the temporary oscillation count. The *D-Flip Flop* samples the output of the *T-Flip Flop* at every positive edge of the inverted *CTRL* signal and the output of this *D-Flip Flop* is the raw random bit.

PUF Mode. In this mode *P/T* is set to 1, thereby breaking the feedback loop. The PUF is enabled via the *TIG* signal which propagates through the two delay chains. We reuse the components of the dual-mode TERO cell and implement a delay based strong PUF. To do this, we divide the number of challenge bits to N -bits and $(64 - N)$ -bits and then give them as inputs to the dual-mode TERO cell and the Auxiliary PUF circuit respectively. The first delay element of the Auxiliary PUF is designed using inverters and the rest use buffers. Finally a *SR* latch constructed using two cross-coupled NAND gates is used to extract the PUF response by comparing the delay of both the chains.

3.4 Linear Delay Model

In this section, we first present the compact representation of the linear delay model for the dual-mode PUF that follows the delay model of a PAPUF [34]. Our target is to model the delay differences between the two branches after the last stage and with the arbiter circuitry excluded. The proposed PUF architecture consists of two non-swapping path based switches as depicted in Figs. 5a & 5b. For a given n -bit challenge c , the delay difference between of the top and bottom paths after the final stage is denoted by $\Delta(n - 1)$. We define the feature vector ϕ for our model by mapping the elements of the challenge vector from $\{0, 1\} \rightarrow \{1, -1\}$. In this case, the PUF response for a given challenge is determined by $s(\Delta(n - 1))$ where $s(x) = 1$ if $x > 0$, $s(x) = 0$ if $x < 0$. For our analysis, we consider the following:

1. w_{i+1} and y_{i+1} indicate the delay components in the upper part of the $(i + 1)^{th}$ stage,
2. x_{i+1} and z_{i+1} represent the delay components in the lower part of the $(i + 1)^{th}$ stage,
3. $\delta_u(i)$ and $\delta_l(i)$ be the propagation delays of the PUF enable a.k.a trigger signal along the upper and lower paths to the input of the $(i + 1)^{th}$ stage respectively.

The propagation delay of the trigger signal for a given feature $\phi \in \{+1, -1\}$ at the output of the $(i + 1)^{th}$ stage is

given by: $2\delta_u(i + 1) = (1 + \phi[i + 1])(\delta_u(i) + w_{i+1}) + (1 - \phi[i + 1])(\delta_u(i) + y_{i+1})$ and $2\delta_l(i + 1) = (1 + \phi[i + 1])(\delta_l(i) + x_{i+1}) + (1 - \phi[i + 1])(\delta_l(i) + z_{i+1})$. Therefore, the delay difference between the upper and lower paths can be estimated as:

$$\begin{aligned} \Delta(i + 1) &= \delta_u(i + 1) - \delta_l(i + 1) \\ &= \Delta(i) + \alpha_{i+1}\phi[i + 1] + \beta_{i+1}, \end{aligned} \quad (1)$$

where $2\alpha_{i+1} = (w_{i+1} - y_{i+1} - x_{i+1} + z_{i+1})$ and $2\beta_{i+1} = (w_{i+1} + y_{i+1} - x_{i+1} - z_{i+1})$. Suppose, $\Delta(-1) = 0$, and then the delay difference $\Delta(n - 1)$ after the final stage can be written as:

$$\Delta(n - 1) = \sum_{i=0}^{n-1} \alpha_i \phi[i] + \sum_{i=0}^{n-1} \beta_i = \sum_{i=0}^n \omega[i] \phi[i] = \langle \omega, \phi \rangle, \quad (2)$$

where $\omega[n] = (\beta_{n-1} + \dots + \beta_0)$ and $\omega[i] = \alpha_i \forall i \in \{0, \dots, n - 1\}$ $\phi[0 : n - 1] = 1 - 2c[0 : n - 1]$, and $\phi[n] = 1$. Thus, the proposed dual-mode PUF follows a linear additive delay model, where:

- the feature vector ϕ is the transformed challenge given by $1 - 2c$, and
- all the elements of ω except $\omega[n]$ depend on the α values and $\omega[n]$ depends on $\beta_0, \dots, \beta_{n-1}$.

It can be observed that the proposed PUF design has an architectural bias due to the $\omega[n] = (\beta_{n-1} + \dots + \beta_0)$. This is because $\omega[n]$ is independent of the challenge bits [34]. As shown in [31], we employ k additional programmable delay elements to reduce the effect of architectural bias. These additional programmable delay elements are referred to as tuning stages. The tuning stages are controlled by independent inputs for the top (*t*) and bottom (*b*) branch and are referred to as tuning bits as shown in Fig. 5c. Considering the last tuning stage as the p^{th} stage ($p = n + k - 1$), the propagation delay at its output is given by:

$$\begin{aligned} 2\delta_u(p) &= (1 + \phi_t[p])(\delta_u(p - 1) + w_p) \\ &\quad + (1 - \phi_t[p])(\delta_u(p - 1) + y_p), \\ 2\delta_l(p) &= (1 + \phi_b[p])(\delta_l(p - 1) + x_p) \\ &\quad + (1 - \phi_b[p])(\delta_l(p - 1) + z_p), \\ \Delta(p) &= \Delta(p - 1) + \beta_p + \frac{\phi_t[p](w_p - y_p) - \phi_b[p](x_p - z_p)}{2} \\ &= \Delta(n - 1) + \sum_{i=n}^p \beta_i + \sum_{i=n}^p \kappa_i, \end{aligned} \quad (3)$$

where $\kappa_i = \frac{\phi_t[i](w_i - y_i) - \phi_b[i](x_i - z_i)}{2}$ represents the tuning variable for reducing the impact of the design induced architectural bias. We can use $\Delta(p) = \langle \omega, \phi \rangle$ by modifying $\omega[n]$ as $\sum_{i=0}^{n-1} \beta_i + \sum_{i=n}^p (\beta_i + \kappa_i)$. Next, to thwart such model building of the proposed PUF design, we introduce two variants of the modified design in the next section.

4 ENHANCED DESIGN WITH RECURRENCE FOR ML RESISTANCE

The vulnerability of Strong PUFs to ML attacks is a known problem and works like [35] derive the theoretical limits of

the learning accuracy using PUF models. Such works are also used to analyze the susceptibility of a PUF design to ML attacks. In this light, one of the design strategies to improve modelling resistance is to obfuscate the challenge by design itself i.e., the basic essence is to ensure that the challenges are unpredictable. This has been demonstrated in popular PUF designs like the Interpose-PUF [2], Feed-Forward APUFs [36] and so on. Recently, a newly proposed PUF design inspired by the Interpose PUF and FF-APUF called the LP-PUF [3] obfuscates its input challenge bits by using mixing operation in its second layer and applying XOR operation in the third layer. In this subsection, we propose a recurrence based challenge obfuscation technique to show greater resistance of the PUF design against ML attacks. This approach exploits the device-specific randomness to encode the challenge, thereby hiding the true relationship between the original challenge and the PUF response. The increased robustness of the recurrence technique was reported in [37]. In light of this, two recurrence schemes for the proposed PUF architecture are presented as: Design-3A and 3B, each utilizing the PUF responses to encode the challenges.

Algorithm 1. Working of Design-3A With θ Cycles

Input: 64 Bit Challenge $\mathbf{c}$, θ
Output: 1 Bit Response
 $k = 0, t = 0, \mathbf{c}_p = \mathbf{c}, \text{nMeas}$
while $t < \theta$ **do**
 while $k < 4$ **do**
 /* Cyclic Left Shift of Challenge */
 $\mathbf{c}_p = \mathbf{c}_p \ll 16$
 $r_k^{(t)} = \text{MV}(\text{PUF}(\mathbf{c}_p), \text{nMeas})$
 $k = k + 1$
 end
 /* $\mathbf{r}_s$ will have the same bit length as $\mathbf{c}$ */
 $\mathbf{r}_s = [r_0^{(t)}, r_1^{(t)}, r_2^{(t)}, r_3^{(t)}, r_0^{(t)}, r_1^{(t)}, r_2^{(t)}, r_3^{(t)}, \dots]$
 $\mathbf{c}_e = \text{XOR}(\mathbf{c}_e, \mathbf{r}_s)$
 $t = t + 1$
 $k = 0$
 $\mathbf{c}_p = \mathbf{c}_e$
end
 /* Final Response */
 $r_0^{(\theta)} = \text{MV}(\text{PUF}(\mathbf{c}_e), \text{nMeas})$

4.1 Design-3A: Enhanced Design With Recurrence Mechanism 1

In this design, we use the idea of encoding the challenge $\mathbf{c}$ with response-bits obtained from the left shifted versions of the same challenge. This process is repeated for θ cycles to recurrently encode the original challenge $\mathbf{c}$. As described in Algorithm 1, we give the challenge $\mathbf{c}$ as input to the PUF in the 0th cycle. Thereafter, we give the encoded $(\mathbf{c} \oplus \mathbf{r}_s)$ or left-shifted challenges $(\mathbf{c} \ll 16p)$, as the case maybe. We denote $\ll 16p$ as the cyclic left shift operation by $16p$ bits with $p \in \{0, 1, 2, 3\}$ and $\mathbf{r}_s$ represents the response vector. The response vector $\mathbf{r}_s$ for the 0th iteration is obtained as $[r_0, r_1, r_2, r_3, r_0, r_1, r_2, r_3, \dots]$ from the Majority Voted PUF (MV-PUF) response for $\mathbf{c}$, $\mathbf{c} \ll 16$, $\mathbf{c} \ll 32$, and $\mathbf{c} \ll 48$. The encoded challenge $\mathbf{c}_e$ given by $(\mathbf{c} \oplus \mathbf{r}_s)$ is used as the input

to the PUF in the next cycle. The final PUF response $r_0^{(\theta)}$ is obtained after θ cycles. A high level recurrence architecture has been shown in Fig. 6a.

Adversary and Threat Model. Here, we consider an adversarial setting for Design-3A where the attacker can tailor the input challenge to obtain the linear PUF response by making the obfuscation due to recurrence trivial i.e., $\mathbf{r}_s$ contains all 0s or all 1s. One such scenario can be thought of where $\mathbf{c}[63 : 48] = \mathbf{c}[47 : 32] = \mathbf{c}[31 : 16] = \mathbf{c}[15 : 0]$ resulting in $r_0 = r_1 = r_2 = r_3$; thereby making the encoded challenge $\mathbf{c}_e = \mathbf{c}$ or $\mathbf{c}_e = \bar{\mathbf{c}}$. Over multiple cycles this will lead to either $r_0^{(\theta)} = \text{MV-PUF}(\mathbf{c})$ (even θ) or $r_0^{(\theta)} = \text{MV-PUF}(\bar{\mathbf{c}})$ (odd θ). However, such challenges occupy only a small portion of the challenge space, $2^{16}/2^{64} \approx 10^{-15}$. Therefore, ruling out such challenges will still leave the PUF with $\approx 2^{63.999}$ challenges which represent almost the entire challenge set. Furthermore, we would like to highlight that such controlled inputs do not help in predicting responses from the random challenges and this has been shown in Section 5.2.1.

4.2 Design-3B: Enhanced Design With Recurrence Mechanism 2

In order to eliminate the possibility for the attacker to devise such controlled inputs as mentioned in the previous sub-section, we improve the feedback mechanism in the proposed Design-3B (Fig. 6b). This is done by ensuring that the response vector $\mathbf{r}_s$ is non-trivial for the attacker to control. We detail the working principle subsequently along with a detailed mathematical analysis to justify the robustness of the design against various possible attacks. Our proposed Design-3B consists of two phases: the *Warmup* and *Evaluation (Eval)* phase. It is to be noted that the *Warmup* phase happens only once before the PUF operation begins. As shown in Algorithm 2, the response vector $\mathbf{r}_s$ is updated in the *Warmup* phase with the MV-PUF responses from 64 predefined randomly generated challenge set, each of 64-bit length. After the *Warmup* phase, we enter the *Eval* phase, where the PUF is queried with a distinct challenge $\mathbf{c}$ from a challenge set $\mathbf{C}_q (N \times 64)$ of cardinality N . The final output corresponding to $\mathbf{c}$ is denoted by $r_0^{(\theta)}$.

4.2.1 Mathematical Analysis

To substantiate the strength of our Recurrence Mechanism 2, we evaluate the bias probability of the final response obtained from Design-3B. We define bias probability as the probability of the linear PUF response r_0 being same as that of the final response $r_0^{(\theta)}$ given by $\Pr(r_0^{(\theta)} = r_0)$. We evaluate this as follows:

$$\begin{aligned} \Pr(r_0^{(\theta)} = r_0) &= \sum_{i=0}^n \Pr(r_0^{(\theta)} = r_0 | |\mathbf{r}_s| = i) \times \Pr(|\mathbf{r}_s| = i) \\ &= \sum_{i=0}^n p_f(i) \times \Pr(|\mathbf{r}_s| = i), \end{aligned} \quad (4)$$

where $|\mathbf{r}_s|$ is the number of ones in the binary representation of $\mathbf{r}_s$ and n denotes the dimension of $\mathbf{r}_s$. The PUF core in Design-3B follows a linear delay model as shown in Section 3.4 and is given by:

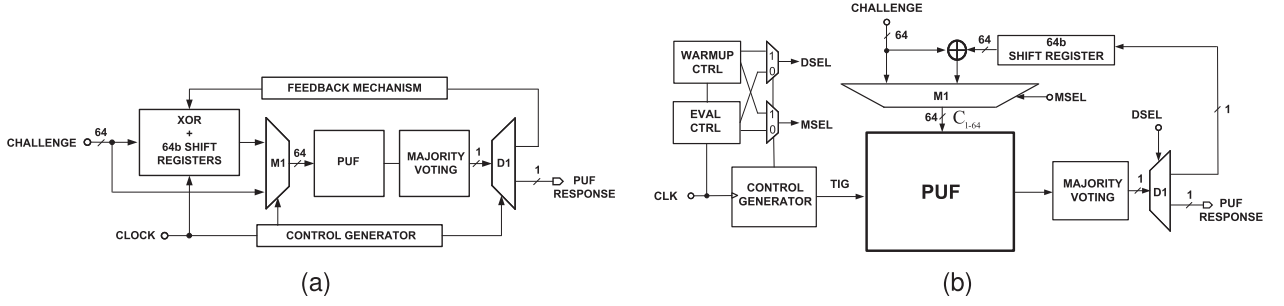


Fig. 6. (a) Block diagram showing the high level behaviour of the proposed PUF with recurrence. (b) Proposed PUF architecture with recurrence.

$$r = s(\langle \omega, \phi \rangle) = \begin{cases} 1, & \text{if } |\langle \omega, \phi \rangle| > 0 \\ 0, & \text{if } |\langle \omega, \phi \rangle| < 0 \end{cases}$$

$$\phi[j] = \begin{cases} 1 - 2c[j], & \forall j \in \{0, 1, \dots, n-1\} \\ 1, & j = n \end{cases}, \quad (5)$$

where ϕ is the transformed challenge vector. In Eq. (4), the bias probability is conditioned on the number of bit-flips in $\mathbf{c}$ which is same as the number of 1s in $\mathbf{r}_s$. For our analysis we consider the parameters of the linear delay model (as shown in Section 3.4) $w_j, y_j, x_j, z_j \forall j \in \{0, 1, \dots, n-1\}$ sampled independently from a normal distribution and represented by i.i.d. random variables (RVs) $W, Y, X, Z \sim \mathcal{N}(0, \sigma^2)$ respectively. Using Eq. (1) we get RVs A, B for α_j, β_j with zero mean and σ^2 variance.

Algorithm 2. Working of Design-3B With θ Cycles

Warm-Up:

Input: Warm-Up Challenge Set $\mathbf{C}_w$

$\mathbf{r}_s = \mathbf{r}_0, n\text{Meas}$

while $k < 64$ **do**

$\mathbf{r}_s = \mathbf{r}_s \ll 1$

$\mathbf{r}_s[0] =$

$\text{MV}(\text{PUF}(\mathbf{C}_w[k, :]), n\text{Meas})$

$k = k + 1;$

end

Eval:

Input: Challenge Set $\mathbf{C}_q (N \times 64)$

Output: 1-Bit Response Set $\mathbf{r}_q (N \times 1)$

$k = 0, t = 0, q = 0$

while $q \leq N$ **do**

$\mathbf{c}_e = \mathbf{c} = \mathbf{C}_q[q, :]$

while $t \leq \theta$ **do**

$\mathbf{r}_s = \mathbf{r}_s \ll 1$

$\mathbf{r}_s[0] = \text{MV}(\text{PUF}(\mathbf{c}_e), n\text{Meas})$

$\mathbf{c}_e = \text{XOR}(\mathbf{c}_e, \mathbf{r}_s)$

$t = t + 1$

end

/* Final Response

$\mathbf{r}_0^{(\theta)} = \text{MV}(\text{PUF}(\mathbf{c}_e), n\text{Meas})$

$\mathbf{r}_q[q] = \mathbf{r}_0^{(\theta)}; q = q + 1$

end

***/**

From Eq. (2), the dot-product $\langle \omega, \phi \rangle$ can be split into $u = \sum_{k \in I} \alpha_k \phi[k]$ and $v = \sum_{k \notin I} \alpha_k \phi[k] + \omega[n]$, where $I = \{q : \mathbf{r}_s[q] = 1\}$ represents the set of indices with bit-flips in the challenge $\mathbf{c}$ or $\mathbf{r}_s[q] = 1$. Note that the sign of u flips when $\phi[k] \forall k \in I$ flips and therefore, the PUF response is determined by comparing the magnitudes of u, v . Let U, V be the random variables representing u, v . In presence of i bit-flips, Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on June 04, 2026 at 09:28:22 UTC from IEEE Xplore. Restrictions apply.

using the expression in [38] we obtain $p_f(i)$ as:

$$p_f(i) = \Pr(|U| < |V|) = 1 - 2/\pi \times \tan^{-1}(\sigma_U/\sigma_V). \quad (6)$$

The variances for U, V can be written as $\text{var}(U) = \text{var}(\sum_{k \in I} \alpha_k \phi[k]) = |I| \text{var}(A) = i\sigma^2$ and $\text{var}(V) = \text{var}(\sum_{k \notin I} \alpha_k \phi[k] + \omega[n]) = (n-i)\sigma^2 + n\zeta\sigma^2$ where

$$\zeta = \frac{\text{var}\left(\sum_{j=0}^{n-1} \beta_j + \sum_{j=n}^p (\beta_j + \kappa_j)\right) + \tau}{\text{var}\left(\sum_{j=0}^{n-1} \beta_j\right)}, \quad (7)$$

$$\tau = \frac{2 \text{Cov}\left(\sum_{k \notin I} \alpha_k \phi[k] \omega[n]\right)}{\text{var}\left(\sum_{j=0}^{n-1} \beta_j\right)}, \quad (8)$$

and κ_j is the tuning variable for the j^{th} stage. Using Eqs. (6), (7), (8) we find the flip probability as:

$$p_f(i) = 1 - \frac{2}{\pi} \tan^{-1}\left(\sqrt{\frac{i}{n\zeta + n - i}}\right). \quad (9)$$

The value of $\sum_{j=n}^p (\beta_j + \kappa_j)$ in $\omega[n]$ depends on the choice of tuning bits [31] and is thereby not independent of $\sum_{j=0}^{n-1} \beta_j$. With increase in the number of tuning elements, ζ reduces because the tuning bits help to compensate the bias term $\sum_{j=0}^{n-1} \beta_j$. The number of 1s in $\mathbf{r}_s$ will follow a binomial distribution with $\text{Bin}(n, p_u)$ where p_u is the PUF core uniformity. In an ideal scenario, $p_u = 0.5$ and $\Pr(|\mathbf{r}_s| = i)$ is evaluated as:

$$\Pr(|\mathbf{r}_s| = i) = \binom{n}{i} \frac{1}{p_u^i (1-p_u)^{n-i}} = \binom{n}{i} \frac{1}{2^n}. \quad (10)$$

Combining Eqs. (9) & (10), we get the bias probability p_b as:

$$p_b = \Pr(r_0^{(\theta)} = r_0) = \sum_{i=0}^n \binom{n}{i} \frac{1}{2^n} \left(1 - \frac{2}{\pi} \tan^{-1}\left(\sqrt{\frac{i}{n\zeta + n - i}}\right)\right). \quad (11)$$

As shown in Fig. 7, Eq. (11) can be visualized in terms of area under the curve for a binomial probability distribution function scaled by $p_f(i)$. In the ideal case for $\zeta = 0$; p_b evaluates to 0.5. However, in practice, with a finite number of tuning elements p_b approaches 0.5.

4.2.2 Reliability Based Modelling: Formulation

Reliability based attacks in the literature have targeted APUFs, XOR APUFs [21], [39] and Interpose-PUFs [39]. For an APUF, if the delay difference between two branches is

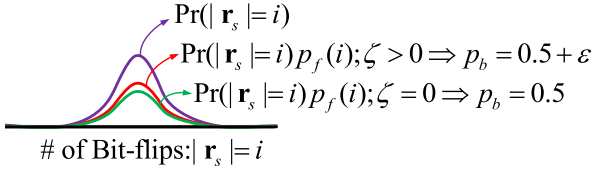


Fig. 7. Bias probability calculation.

less than a threshold then the response is termed unreliable due to the presence of process variations. Evolution Strategies (ES) such as CMA-ES [21] find the best estimate of ω by maximizing the correlation between the measured reliability h_i and hypothetical reliability $\tilde{h}_i$. The measured reliability h_i is given by $|l/2 - \sum_{j=1}^l r_{i,j}|$ where, l is the number of measurements. The attack computes a hypothetical reliability $\tilde{h}_i$ with a threshold γ given by:

$$\tilde{h}_i = \begin{cases} 1, & \text{if } |\langle \omega, \phi_i \rangle| > \gamma \\ 0, & \text{if } |\langle \omega, \phi_i \rangle| \leq \gamma \end{cases} \quad (12)$$

In [39], the authors propose a variant of reliability attack well-suited for composite PUF designs. The objective of the reliability attack for our Design-3B is to learn the behaviour of a single PUF instance. However, due to the presence of recurrence scheme, the hypothetical reliabilities cannot be directly computed for a given challenge using Eq. (12). Therefore, we next discuss how the reliability attack in [21] can be adapted for Design-3B.

Initially, ω is set to ω_{random} . Response vector $\mathbf{r}_{s,0}$ and the measured reliability vector $\mathbf{h}_{r,0}$ after the *warmup* phase is used to find the hypothetical response vector $\tilde{\mathbf{r}}_{s,i}$ and its corresponding hypothetical reliability vector $\tilde{\mathbf{h}}_{r,i}$ for each feature ϕ_i . This is done as follows:

$$\begin{aligned} \tilde{\mathbf{h}}_{r,i} &= \{\tilde{h}_{r,i-1} \ll 1, (|\langle \omega_{\text{random}}, \phi_i \rangle| > \gamma)\} \forall i \in \{1, 2, \dots, N\}, \\ \tilde{\mathbf{r}}_{s,i} &= \{\tilde{r}_{s,i-1} \ll 1, (\langle \omega_{\text{random}}, \phi_i \rangle > 0)\} \forall i \in \{1, 2, \dots, N\}. \end{aligned} \quad (13)$$

The input challenge $\mathbf{c}_i$ in the *Eval* phase is encoded using:

- 1) $(1 - \tilde{\mathbf{h}}_{r,i})$: accounts for the unreliable bits leading to the occurrence of flips and,
 - 2) $\tilde{\mathbf{r}}_{s,i}$: considers the XOR operation with response bits.
- This can be written as:

$$\begin{aligned} \mathbf{c}_i &= \mathbf{c}_i \oplus \tilde{\mathbf{r}}_{s,i} \oplus (1 - \tilde{\mathbf{h}}_{r,i}), \\ \hat{h}_i &= |\langle \omega_{\text{random}}, \phi_i \rangle| > \gamma. \end{aligned} \quad (14)$$

CMA-ES algorithm updates the value of ω_{random} till the correlation between $\mathbf{h} = [h_1, h_2, \dots]$ and $\hat{\mathbf{h}} = [\hat{h}_1, \hat{h}_2, \dots]$ crosses a threshold, where $\hat{h}_i$ is the estimated hypothetical reliability value for feature ϕ_i .

The next section provides a deeper look into the implementation details, TRNG bitstream evaluation and validates that Design 3 not only satisfies standard PUF metrics but also shows strong robustness to ML/DL based modelling and reliability attacks.

5 IMPLEMENTATION AND SECURITY EVALUATION

In this section, we implement a FPGA prototype of the proposed architecture and validate the TRNG and PUF quality metrics. We also show that the PUF instance (Design-3B)

has strong resistance to reliability attacks as well as state-of-the-art ML/DL based modelling attacks. We develop the proposed architecture by utilizing Verilog as the hardware description language in the Xilinx Vivado (v2019.1) design environment and evaluate using seven instances of the Artix-7 (XC7A100T) FPGA on Nexys-4 DDR board at normal operating conditions (see Fig. 10 for the implementation prototype). All the logic in the the dual-mode TERO cell are realized by direct instantiation of the LUT6 primitive in slices “SLICEL” of the FPGA. We implement the stages by two independent chains using the Programmable Delay Line (PDL) as shown in [31]. The PDL implementation takes advantage of the internal structure of a LUT6 and each LUT6 is configured to behave as a delay element. Next, to reconfigure the dual-mode TERO cell, we ensure that the input signal is connected to a specific port of the LUT6 primitive depending on the PUF/TRNG mode of operation. Fig. 8 illustrates a closer look into the implementation specifics. Such reconfiguration ensures that two propagation paths are chosen within a stage for each PDL depending on the mode of operation. The control signal during TRNG operation is generated using a RO oscillating at around 155MHz connected to a 9-bit synchronous counter. However, to achieve a sufficient entropy rate, the TERO loop must be unbalanced in a way that the number of oscillation periods denoted by M fulfils the following criteria [33]:

$$100 < M < \frac{T_{\text{meas}}}{T_{\text{osc}}}, \quad (15)$$

where T_{meas} is portion of the control period during which the dual-mode TERO cell is enabled and T_{osc} is the oscillation period. In other words, a huge asymmetric TERO cell would lead to the oscillator not oscillating for a sufficiently long time, thereby the entropy would be too small. On the contrary, a perfectly symmetrical TERO cell would lead to oscillations lasting longer than the control period resulting in non-random counter values. As the above two situations are undesirable, we avoid these cases by starting with a large number of elements in each TERO branch (in our case, we started with 13 delay elements and a NAND gate) and check the counter value distribution. Depending on this distribution of counter values, we gradually reduce the number of delay elements in smaller steps such that Eq. (15) is satisfied. It is difficult to obtain this repeatedly, since devices configured with the same bit-stream can give very undesirable results. This phenomena is further explained in Fig. 9. We show the results from different placements across different FPGA boards. The location of the PUF/TRNG MUX is used as a reference. Results from the same placement location on different boards (refer to Figs. 9a & 9b) as well as different placements on the same board (refer to Figs. 9c & 9d) are presented. We see that the slice position X16Y43 on Board 1 and Board 4 indicate a set of positions where the implemented TERO cell satisfies the above condition and has more than 100 transitions.

Such a position is labelled as a “good position”. On the other hand, the slice position X16Y43 on Board 3 represents an extreme set of positions with only 4 possible transition values and is therefore deemed as “unsuitable”. Therefore, we focus our implementation of the proposed dual-mode

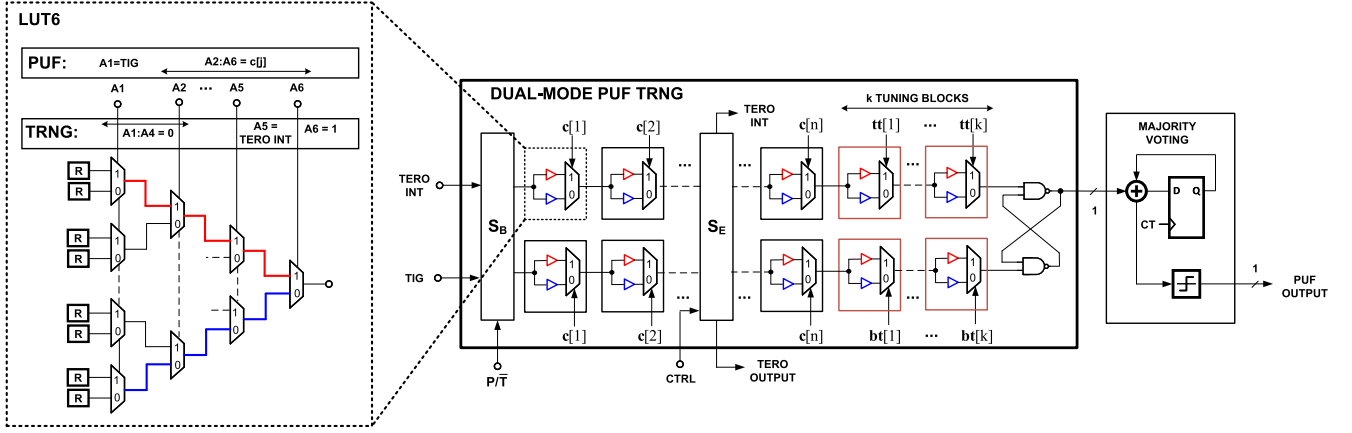


Fig. 8. Implementation of unified PUF-TRNG architecture with majority voting.

TERO cell after experimentally identifying a single good position in all the devices under test and program them with the same bitstream. Our reference position X40Y51 has a performance histogram approximating normal distribution.

Next, we extend our dual-mode TERO cell to an unified PUF-TRNG by implementing another 57 delay elements in both the delay chains of the proposed auxiliary PUF circuit. Additionally, there is a fixed number of tuning stages similar to the PUF stages [31]. Each tuning stage is realized by two independent control bits that compensate for the bias induced between both the delay chains. In addition to this core, the design contains a control signal generator for enabling the circuit, a majority voting block to ensure generation of reliable PUF response, separate bit extraction logic for its operation as a PUF or TRNG, control logic and Universal Asynchronous Receiver/Transmitter (UART) for communication. Finally, to reduce the noise due to these additional blocks, the PUF instance is implemented inside a pblock (a physical block that restricts the logical block to a specific area of the FPGA), where implementation of other parts are prohibited.

Timing Details. The Warmup phase takes $244 \mu s$ to prepare the response vector $\mathbf{r}_s$ for the Eval phase. The time

taken to generate a single PUF response from Design-3B during the Eval phase is $7.6 \mu s$. Conversely, the TRNG produces a throughput of 1.25 Mbps. In regard to other design parameters, the TRNG operates at a frequency of 100 MHz while the PUF operates at a clock frequency of 28.6 MHz.

5.1 TRNG Evaluation

Next we evaluate the quality of the designed TRNG circuit. To measure the uniformity of the random numbers generated by the TRNG i.e. the probability of generating '1' equals the probability of generating '0', we evaluate the min-entropy. **Min – entropy** is calculated as follows:

$$\text{Min – entropy} = -\log_2 \max\{p_0, p_1\}, \quad (16)$$

where p_0 and p_1 represent the probabilities of generating 0 & 1 respectively.

Fig. 12a shows speckle pattern for 512 randomly selected 512-bit key, with black representing '1' and white representing '0' from a single FPGA. We observe that the number of 1s & 0s follow a homogeneous distribution, and the calculated min-entropy is 0.999902 for 100M samples, thereby indicating excellent cryptographic qualities of the generated bit-stream. Another important feature of a random bit-stream is independence i.e., the future outputs are not impacted by the current output. This feature can be validated by the auto-correlation test, which computes the correlation coefficient between original sequence and lagging sequence [40]. Auto-correlation with lag k for a sequence of length N is defined as:

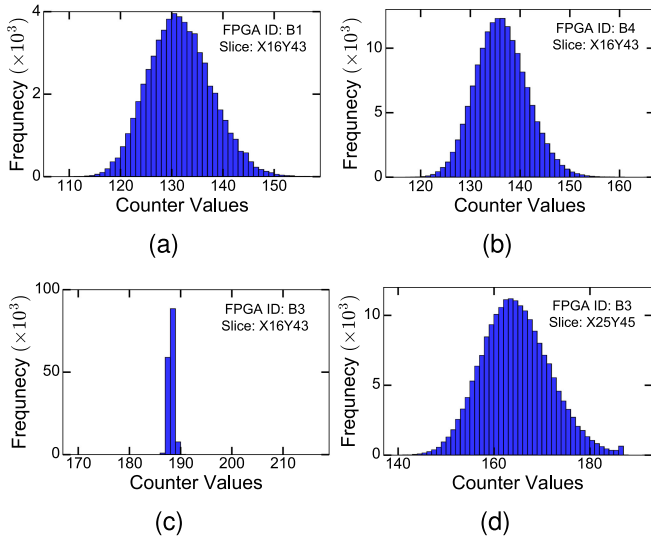


Fig. 9. Results of the oscillation frequency in (a), (b), (d) good, and (c) unsuitable location on separate FPGAs.

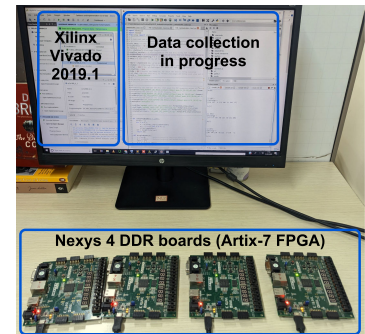


Fig. 10. Implementation setup and prototype.

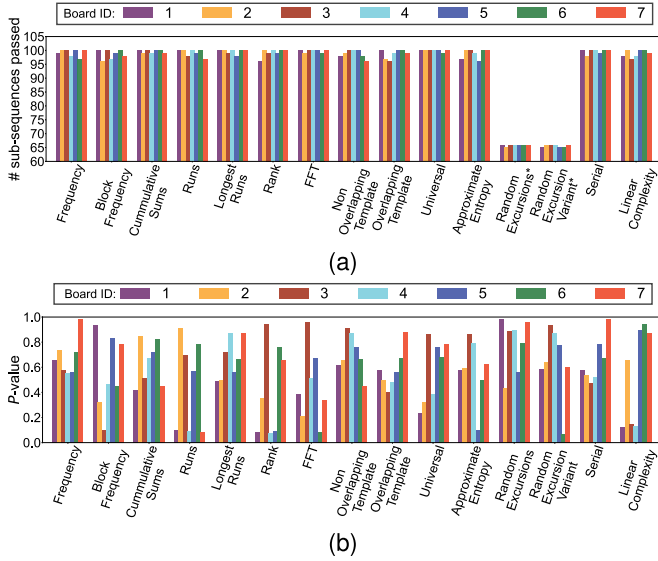


Fig. 11. NIST SP800-22 test results depicting board-wise (a) pass rate*, and (b) *p-value*.

$$\text{ACF}(k) = R_{xx}(k) = \frac{\sum_{i=0}^{N-k-1} (x_i - \mu)(x_{i+k} - \mu)}{N * \sigma^2}, \quad (17)$$

where $\mathbf{x}$ is the TRNG sequence and μ, σ^2 are the mean and the variance of the sequence. Fig. 12b shows the auto-correlation test result again showing high-quality, almost ideal randomness. Furthermore, the TRNG output should be consistent over time and across different realizations. To ensure this, we verify the wide sense stationarity (WSS) property for our bit-stream. WSS processes require only the mean and the auto-correlation to be constant over all samples. We first find the power spectral density (PSD) of our TRNG output and then the variance of PSD over all the frequency windows.

$$S_{xx}(f) = \mathcal{F}(R_{xx}) = \sum_{i=-N+1}^{N-1} R_{xx} e^{-2\pi f k i}. \quad (18)$$

The variance in Fig. 12c is lesser than a user-defined value σ_0 , signifying that the TRNG output satisfies the WSS [40] property. Lastly, we evaluate the statistical properties of the generated bitstream against standard test suites like the NIST SP800-22 and AIS-31. Each statistical test of the NIST SP800-22 outputs a *p-value* depending on the proportion of sub-sequences that have passed the test. The NIST SP800-22 tests were conducted with default parameters over 10^8 raw random bits for our analysis. These random bits were divided into 100 sub-sequences, each of length 10^6 . Furthermore, for verification purposes, sequences of raw bits were generated and tested using the tests proposed in the AIS-31 suite. The NIST SP800-22 and AIS-31 test results for all the seven boards are presented in Fig. 11 and Table 1 respectively. As observed, the raw random numbers pass all the tests, thereby demonstrating excellent statistical randomness.

5.2 PUF Quality Analysis

We evaluate our proposed PUF architecture without recurrence (Design-2) on all seven FPGA boards with 400K random challenges. The golden responses were estimated

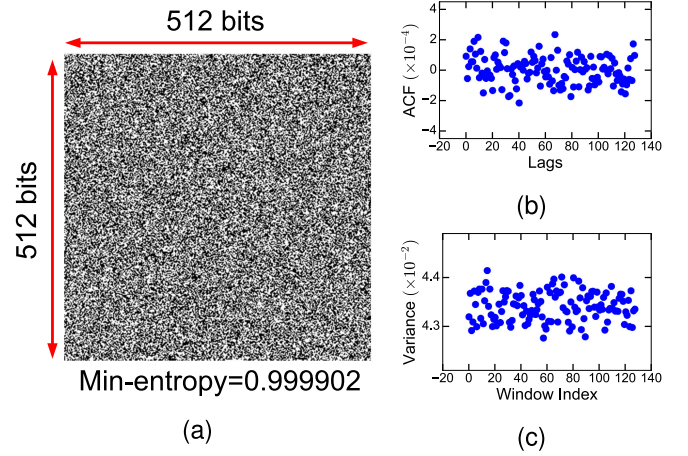


Fig. 12. The (a) speckle pattern of the generated random bitstream (black = “1”, white = “0”), (b) measured ACF for TRNG output, and (c) measure of the variance of the PSD of the TRNG output for stationarity.

using majority voting over 15 measurements at nominal operating conditions. The uniqueness, average uniformity and reliability across the seven boards is calculated to be 47.25%, 48.57% and 97.63% respectively. The reported values are close to their ideal values.

Next, we evaluate both Design-3A and 3B. To do this, we take four boards and characterize the PUF instance with 5.2M challenges. We again estimate the golden response using majority voting over 15 evaluations at normal operating conditions. The average uniformity, uniqueness and reliability values for Design-3A and 3B are (49%, 50.4%), (47.6%, 48.65%), and (96.59%, 98.29%) respectively as furnished in Table 2. However, when it comes to enrolling the PUF, we believe standard enrollment schemes like Secure Sketch [41] can be adopted to our proposed PUF owing to its high reliability.

5.2.1 Robustness to Modelling and Reliability Attacks

Modelling Attacks. As shown in Section 3.4, the proposed PUF architecture without recurrence is susceptible to ML based modelling attacks due to its linear additive delay model. In order to support this claim we train linear ML models such as Linear Regression (LR) with RProp and Linear Support Vector Machine (LSVM) with 90K CRPs and test on 10K CRPs each of which reports an accuracy of 94.8%, 94.92% respectively. One may note that ML/DL modelling attacks should be performed on a PUF instance with desirable properties. We choose the PUF instance with uniformity of 50.1% corresponding to Design-3B for performing modelling attacks. To do this, we have collected 5200K CRPs and split it into 90:10 train-test ratio. We have performed 5-fold cross validation for all the algorithms. Fig. 13b shows a maximum accuracy of close to 50% with all the ML and DL models. This corresponds to not only a significant reduction in accuracy when compared to that of the PUF architecture without recurrence but also close to the ideal value of 50%.

* The *Random Excursions* and the *Random Excursion Variant* tests need large amount of data in comparison to other tests and thus have 66 sequences corresponding to them [27].

TABLE 1
Board-Wise AIS-31 Test Results on TRNG Output

Test Name	FPGA board ID							Result
	1	2	3	4	5	6	7	
Procedure A								
T0-T5	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS
Procedure B								
T6	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS
(<i>d</i> < 0.025)	0.00008	0.00293	0.0003	0.00032	0.00103	0.001	0.001980	
(<i>s</i> < 0.02)	0.00108	0.0022	0.00219	0.00193	0.00264	0.000980	0.000260	
T7	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS
(<i>s</i> ₁ < 15.31)	0.10368	0.13448	0.08450	0.15488	1.34163	1.479684	0.94178	
(<i>s</i> ₂ < 15.13)	0.00135	1.63593	0.00072	0.00512	1.56802	0.023120	0.557781	
T8	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS
(<i>s</i> > 7.976)	8.11469	8.11109	8.11228	8.10884	8.11049	8.10949	8.11136	

On further analysis, we see that the prediction accuracy with more sophisticated ML Algorithms such as SVM with Polynomial and RBF kernels is close to 52.27%. We, then attack the PUF instance with Random Forest (RF) Classifier (no. of estimators =150) and obtain an accuracy of 51.7%. Finally, we check the resistance of the design against Deep Neural Network (DNN) composed of 4 fully connected hidden layers, each having 4096 neurons, and constructed using the TensorFlow library. The model is trained using ‘categorical cross-entropy loss’ with the ‘adam’ optimization algorithm to obtain an accuracy of 52.53%. The prediction accuracy for different number of CRPs and different models has been illustrated in Fig. 13b.

For modelling attacks on PUF instances, there can be multiple implementations, but not all of them work the same. Therefore, we use the PyPUF library’s [29] attack models to have a fair comparison of the results. We have evaluated our proposed design against the PyPUF’s Linear Regression, Logistic Regression as well as the Multilayer Perceptron (MLP) attacks and have obtained an accuracy of 53.12%, 52.064% and 55.37% respectively. This has been illustrated in Fig. 13c. We observe similar trends in the accuracy values between our ML model implementations and the PyPUF attack models. This validates the generality as well as correctness of our attack mechanism. Furthermore, all the reported accuracy values are close to the ideal value of 50%. Therefore, Design-3B can be said to have strong resilience against state-of-the-art modelling attacks.

Reliability Attacks. We analyse the vulnerability of Design-3B to reliability based ML attacks following the strategy presented in Section 4.2.2. We collect 5.2M CRPs with 5 measurements to calculate the measured reliability of target PUF with obfuscation.

The adopted version of CMA-ES algorithm is used to train over 4.68M and test over 0.52M samples. From Fig. 14a, it is observed that the correlation between measured and hypothetical reliability saturates at 0.0838 after 75K iterations with an accuracy of 50.14%.

Chosen Challenge Attacks. Section 4.1 mentions that the objective of an adversary is to predict the response of a randomly generated challenge by applying a set of tailored challenges best suited for the obfuscation scheme under analysis (Design-3A). To do this, we consider all the possible 65,536 CRPs in the training set and 13K randomly generated challenges as the test set. The prediction accuracy was reported to be 62.2% (see Fig. 14b). Hence, such controlled inputs do not help in predicting the responses from randomly generated challenges. Furthermore, our final design (Design-3B) is agnostic to any kind of structural attacks as the contents of the response vector is always non-trivial.

Although ML experiments *can disprove* security claims but *cannot prove* any security claim. We improve the confidence of our Design-3B by demonstrating the prediction accuracy on 5.2M CRPs, which is the highest number collected from hardware in the literature to the best of our knowledge. The prediction accuracy is close to the ideal value of 50%; thereby showing strong resistance to modelling attacks.

5.3 Comparison With State-of-the-Art

In Table 3, we perform a comparison of the hardware footprint and throughput of our architecture with the state-of-the-art standalone PUFs/TRNGs as well as Integrated PUF-TRNG

TABLE 2
Quality Metrics (%) of Design-2, 3A, 3B on FPGAs

PUF Metrics	Design-2 FPGA Board ID							Design-3A FPGA Board ID				Design-3B FPGA Board ID			
	1	2	3	4	5	6	7	1	2	3	4	1	2	3	4
Uniformity	47.14	50.43	48.32	49.16	51.23	46.45	47.28	47.52	51.34	47.62	49.54	52.17	50.10	47.20	52.22
Uniqueness				47.25						47.60				48.65	
Reliability	98.68	98.91	97.12	98.00	97.00	97.54	96.16	98.23	96.78	96.12	95.25	98.78	98.97	97.15	98.28

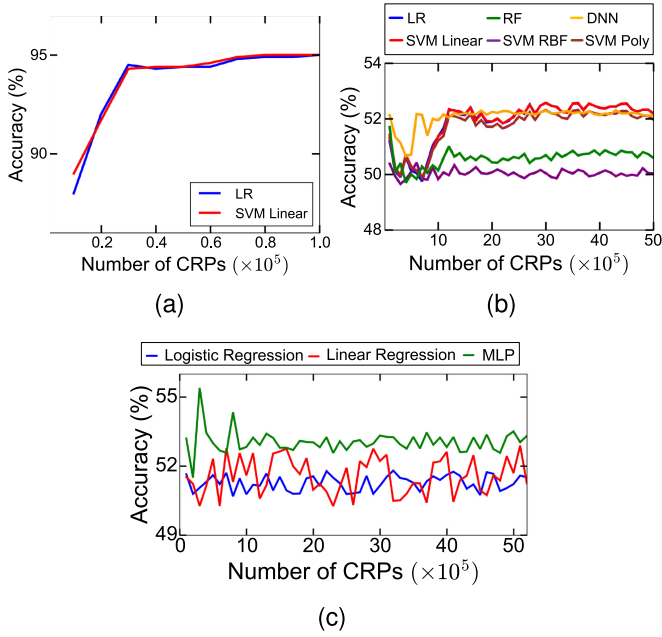


Fig. 13. Modelling accuracy using (a) LR, linear SVM (without recurrence), (b) LR, linear SVM, SVM with RBF and poly kernels, random forest, and DNN (with recurrence), and (c) PyPUF models like linear, logistic regression and MLP.

designs. We see a whopping 48.62% reduction in area is achieved by our Integrated PUF-TRNG (Design-2) compared to separate PDL based APUF and TERO-TRNG implementations. We would also like to mention that the resource utilization and delay of the critical path of the logic circuit depends on the number of inputs going into the LUT [42]. Therefore, the designs in our comparison table utilizing a LUT6 are compatible for comparisons based on area or hardware footprint. Furthermore, our PUF-TRNG hardware cost scales linearly with challenge size, unlike exponential scaling observed in [15], [16], [13]. Lastly, one may note that some of the fields in Table 3 denote N/A. It can be explained as: 1) *Challenge Size*: This input parameter is applicable only in the case of PUF designs (standalone or integrated) and not for TRNGs, 2) *Throughput*: The throughput of a PUF and TRNG is reported separately in the comparison table, therefore, the TRNG throughput is N/A in the case of TERO-PUF and the PUF

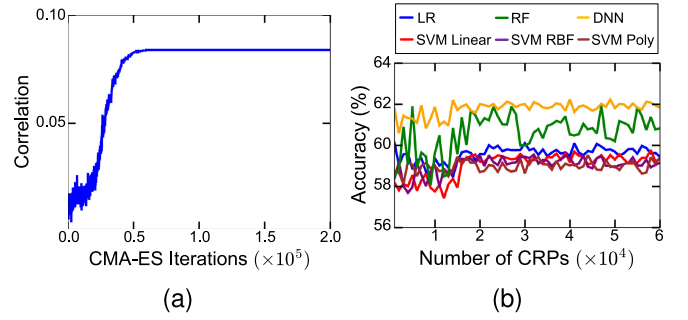


Fig. 14. (a) CMA-ES based reliability attack. (b) Chosen challenge attack.

throughput is reported as N/A in the case of TERO-TRNG, 3) *Modelling Accuracy*: For key generation PUFs (also known as Weak PUFs) such as TERO-PUF [5] as well as integrated weak PUF-TRNG designs [13], the hardware footprint grows exponentially with the challenge length. This attribute makes them suitable for its use as key generators. Furthermore, the challenge length is not large enough, thereby, making the modelling of such architectures not directly relevant.

Table 4 compares the robustness of our Design-3B against ML and reliability based attacks with state-of-the-art PUF constructions. It can be observed that our proposed design has much stronger resistance than state-of-the-art PUF constructions against modelling attacks. One may note that the Rec-DAPUF [37] construction achieves a moderate modelling accuracy of 64.9% using SVM with only 50K CRPs; which is a relatively modest size in the context of machine learning PUFs. In contrast, we not only demonstrate our modelling attacks on 5.2M CRPs ($\approx 2^{23}$) and achieve a close to ideal accuracy value of 55.37% but also perform a tailored version of reliability attack for our construction and obtain a corresponding accuracy of 50.14%.

6 DISCUSSION

Resilience to attacks combining the partial knowledge from both the TRNG and PUF design: We want to emphasize on the fact that the essential crux of our architecture from the design perspective lies in the utilization of two very different entropy sources (metastability for TRNG and propagation delay for PUF) to generate the TRNG bitstream and the PUF

TABLE 3
Comparison Table for Hardware Resource Utilization and Performance

	Challenge Size	LUTs	FFs	F5 MUXes	Tech	Entropy Source	Throughput-(Mbps)		Modelling Accuracy
							TRNG	PUF	
Standalone PUFs/TRNGs									
TERO-PUF [5]	8	3072	190	0	FPGA (Spartan-6)	Metastability	N/A	NR	N/A
TERO-TRNG [33]	N/A	39	12	0	FPGA (Spartan-6)	Metastability	0.625	N/A	N/A
PDL based APUF [31]	64	276	0	0	FPGA (Virtex-5)	Delay	N/A	NR	-
Our Work (Design-2)	64	152	16	0	FPGA (Artix-7)	Metastability & Delay	1.25	0.31	-
Integrated PUF-TRNG Designs									
Varchola et al. [14]	-	1296	839	0	FPGA (Actel-Fusion)	Metastability	0.9	NR	N/A
Maiti et al. [13]	7	1288	945	0	FPGA (Spartan-3)	Jitter	3.2	NR	N/A
Our Work (Design-3B)	64	618	447	0	FPGA (Artix-7)	Metastability & Delay	1.25	0.31	52.53%

NR - Not Reported; N/A - Not Applicable.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on June 04, 2026 at 09:28:22 UTC from IEEE Xplore. Restrictions apply.

TABLE 4
Comparison Table for Modelling Accuracy Across Several PUF Designs

		APUF	4XORs	5XORs	4XORs	(1-4)	Rec	Our
		[20]	APUF	APUF	LSPUF	IPUF	DAPUF	Work
		[2]	[20]	[20]	[20]	[39]	[37]	
		[43]	[2]	[2]	[43]			
				[43]				
Stages		64						
Prediction Accuracy (in %)	LR	99.90	99.00	99.00	99.00	99.00	56.90	52.51
	SVM	99.00	-	-	-	-	64.90	52.27
	DNN	99.50	94	98.20	98.50	-	-	52.53
	Reliability Attack	99.00	97.60	96.20	-	87.00	-	50.14
	MLP	-	95.00	95.00	-	-	-	55.37
	LR	18.05	12	80	12	200	50	5200
CRPs ($\times 1000$)	SVM	3.25	-	-	-	-	50	5200
	DNN	6.80	30	145	240	-	-	5200
	Reliability Attack	20	150	200	-	40	-	5200
	MLP	-	150	200	-	-	-	5200

response. From the implementation perspective, it is well known that different pins of the LUT6 primitive have asymmetric delays. We use pin A1 as input and A2-A6 as challenge bits during PUF operation; whereas pin A5 is used as the input pin for TRNG implementation. Next, the number of stages in both the modes are different [(7 and (7 + 57) stages for TRNG and PUF mode of operation respectively)]. It is to be noted that during the TRNG mode of operation, the time duration after which the TERO cell restarts varies depending on the periodicity of the RO in the control signal generator. Therefore, from an adversarial outlook, knowledge about the number of oscillations in the dual-mode TERO cell will not aid in the estimation of the propagation delay of the PUF circuitry. Similarly, information about the propagation delay of the PUF circuitry will not help in estimating the number of temporary oscillations of the dual-mode TERO cell. However, additional investigation is required to substantiate the same and can be considered as a future direction of work.

Preliminary Robustness Analysis of the proposed design: For analyzing the temperature sensitivity of our proposed PUF design in the integrated PUF/TRNG primitive, we randomly generated 20k CRPs and estimated the golden response using majority voting over 5 measurements across the temperature spectrum -25°C to $+70^{\circ}\text{C}$. Fig. 15 shows that under extreme operating conditions, approximately 92% of the CRPs give reliable response when measured to

the golden response obtained at 25°C . There is an opportunity of estimating the responses at different operating point from the ambient temperature using ML techniques [44]. Leveraging this prior research and using them to tune Error Correcting Code parameters for further boosting the reliability can be considered as a future scope of work.

7 CONCLUSION

This paper initiated the study of a dual-mode candidate Strong PUF-TRNG architecture co-residing in a single circuit using two different hardware-intrinsic properties. We analyse the robustness of the proposed design against ML and DL attacks and then proposed an enhanced circuit design with recurrence that shows strong resistance to modelling and reliability based attacks. Our proposed scheme achieved desirable security properties by passing statistical tests such as NIST SP800-22 and AIS31 for TRNGs. On the other hand, experimental implementations deployed on Artix-7 FPGA are furnished to show the less hardware footprints of the designs. We leave it open to study the impact of environmental influences on the proposed design and the implementation of the architecture on an ASIC platform.

REFERENCES

- [1] B. Gassend, D. Clarke, M. V. Dijk, and S. Devadas, "Silicon physical random functions," in *Proc. 9th ACM Conf. Comput. Commun. Secur.*, 2002, pp. 148–160.
- [2] P. H. Nguyen, D. P. Sahoo, C. Jin, K. Mahmood, U. Rührmair, and M. van Dijk, "The interpose PUF: Secure PUF design against state-of-the-art machine learning attacks," *Cryptol. ePrint Arch.*, 2018.
- [3] N. Wisiol, "Towards attack resilient arbiter PUF-based strong PUFs," *Cryptology ePrint Archive*, Paper 2021/1004, 2021. [Online]. Available: <https://eprint.iacr.org/2021/1004>
- [4] A. Maiti and P. Schaumont, "Improved ring oscillator PUF: An FPGA-friendly secure primitive," *J. Cryptol.*, vol. 24, no. 2, pp. 375–397, 2011.
- [5] L. Bossuet, X. T. Ngo, Z. Cherif, and V. Fischer, "A PUF based on a transient effect ring oscillator and insensitive to locking phenomenon," *IEEE Trans. Emerg. Topics Comput.*, vol. 2, no. 1, pp. 30–36, Mar. 2014.

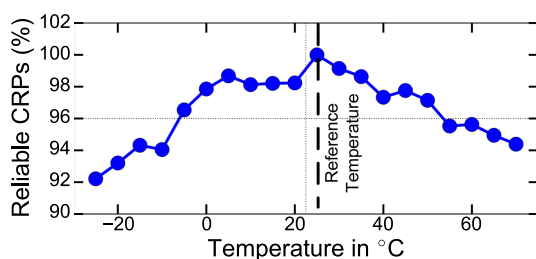


Fig. 15. PUF response against varying temperature.

- [6] Z. Cherif, J. Danger, S. Guilley, and L. Bossuet, "An easy-to-design PUF based on a single oscillator: The loop PUF," in *Proc. IEEE 15th Euromicro Conf. Digit. Syst. Des.*, 2012, pp. 156–162.
- [7] K. Xiao, M. T. Rahman, D. Forte, Y. Huang, M. Su, and M. Tehranipoor, "Bit selection algorithm suitable for high-volume production of SRAM-PUF," in *Proc. IEEE Int. Symp. Hardware-Oriented Secur. Trust*, 2014, pp. 101–106.
- [8] B. Jun and P. Kocher, "The Intel random number generator," *Cryptogr. Res. Inc. White Paper*, vol. 27, pp. 1–8, 1999.
- [9] S. Robson, B. Leung, and G. Gong, "Truly random number generator based on a ring oscillator utilizing last passage time," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 61, no. 12, pp. 937–941, Dec. 2014.
- [10] E. N. Allini, O. Petura, V. Fischer, and F. Bernard, "Optimization of the PLL configuration in a PLL-based TRNG design," in *Proc. Des. Autom. Test Eur. Conf. Exhib.*, 2018, pp. 1265–1270.
- [11] K. Yang, D. Fick, M. B. Henry, Y. Lee, D. Blaauw, and D. Sylvester, "A 23Mb/s 23pJ/b fully synthesized true-random-number generator in 28nm and 65nm CMOS," in *Proc. IEEE Int. Solid-State Circuits Conf. Dig. Tech. Papers*, 2014, pp. 280–281.
- [12] A. Cherkaoui, V. Fischer, A. Aubert, and L. Fesquet, "A self-timed ring based true random number generator," in *Proc. IEEE 19th Int. Symp. Asynchronous Circuits Syst.*, 2013, pp. 99–106.
- [13] A. Maiti, R. Nagesh, A. Reddy, and P. Schaumont, "Physical unclonable function and true random number generator: A compact and scalable implementation," in *Proc. 19th ACM Great Lakes Symp. VLSI*, 2009, pp. 425–428.
- [14] M. Varchola, M. Drutarovsky, and V. Fischer, "New universal element with integrated PUF and TRNG capability," in *Proc. Int. Conf. Reconfigurable Comput. FPGAs*, 2013, pp. 1–6.
- [15] S. K. Satpathy et al., "An all-digital unified physically unclonable function and true random number generator featuring self-calibrating hierarchical Von Neumann extraction in 14-nm tri-gate CMOS," *IEEE J. Solid-State Circuits*, vol. 54, no. 4, pp. 1074–1085, Apr. 2019.
- [16] M. Danesh et al., "Unified analog PUF and TRNG based on current-steering DAC and VCO," *IEEE Trans. Very Large Scale Integr. Syst.*, vol. 28, no. 11, pp. 2280–2289, Nov. 2020.
- [17] M. Majzoobi, F. Koushanfar, and S. Devadas, "FPGA-based true random number generation using circuit metastability with adaptive feedback control," in *Proc. Int. Workshop Cryptographic Hardware Embedded Syst.*, 2011, pp. 17–32.
- [18] U. Rührmair, F. Sehnke, J. Sölter, G. Dror, S. Devadas, and J. Schmidhuber, "Modeling attacks on physical unclonable functions," in *Proc. 17th ACM Conf. Comput. Commun. Secur.*, 2010, pp. 237–249.
- [19] J. Delvaux, "Machine-learning attacks on PolyPUFs, OB-PUFs, RPUFs, LHS-PUFs, and PUF-FSMs," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 8, pp. 2043–2058, Aug. 2019.
- [20] U. Rührmair et al., "PUF modeling attacks on simulated and silicon data," *IEEE Trans. Inf. Forensics Security*, vol. 8, no. 11, pp. 1876–1891, Nov. 2013.
- [21] G. T. Becker, "The gap between promise and reality: On the insecurity of XOR arbiter PUFs," in *Proc. Int. Workshop Cryptographic Hardware Embedded Syst.*, 2015, pp. 535–555.
- [22] T. Kroeger, W. Cheng, S. Guilley, J. L. Danger, and N. Karimi, "Effect of aging on PUF modeling attacks based on power side-channel observations," in *Proc. Des. Autom. Test Eur. Conf. Exhib.*, 2020, pp. 454–459.
- [23] F. Ganji, S. Tajik, and J.-P. Seifert, "PAC learning of arbiter PUFs," *J. Cryptographic Eng.*, vol. 6, no. 3, pp. 249–258, 2016.
- [24] M. Majzoobi, M. Rostami, F. Koushanfar, D. S. Wallach, and S. Devadas, "Slender PUF protocol: A lightweight, robust, and secure authentication by substring matching," in *Proc. IEEE Symp. Secur. Privacy Workshops*, 2012, pp. 33–44.
- [25] A. Aghaie and A. Moradi, "TI-PUF: Toward side-channel resistant physical unclonable functions," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 3470–3481, Apr. 2020.
- [26] S. Burman, D. Mukhopadhyay, and K. Veezhinathan, "LFSR based stream ciphers are vulnerable to power attacks," in *Proc. Int. Conf. Cryptol. India*, 2007, pp. 384–392.
- [27] A. Rukhin et al., "SP 800–22 Rev. 1a. A statistical test suite for random and pseudorandom number generators for cryptographic applications," Gaithersburg, MD, USA, 2010.
- [28] W. Killmann and W. Schindler, "A proposal for: Functionality classes for random number generators," Univ. Bonn, Bonn, Germany, 2011.
- [29] N. Wisiol et al., "pypuf: Cryptanalysis of physically unclonable functions," 2021. [Online]. Available: <https://doi.org/10.5281/zenodo.3901410>
- [30] M. Varchola and M. Drutarovsky, "New high entropy element for FPGA based true random number generators," in *Proc. Int. Workshop Cryptographic Hardware Embedded Syst.*, 2010, pp. 351–365.
- [31] M. Majzoobi, F. Koushanfar, and S. Devadas, "FPGA PUF using programmable delay lines," in *Proc. IEEE Int. Workshop Inf. Forensics Secur.*, 2010, pp. 1–6.
- [32] V. Rožić, B. Yang, J. Vliegen, N. Mentens, and I. Verbauwhede, "The Monte Carlo PUF," in *Proc. IEEE 27th Int. Conf. Field Programmable Log. Appl.*, 2017, pp. 1–6.
- [33] O. Petura, U. Mureddu, N. Bochard, V. Fischer, and L. Bossuet, "A survey of AIS-20/31 compliant TRNG cores suitable for FPGA devices," in *Proc. IEEE 26th Int. Conf. Field Programmable Log. Appl.*, 2016, pp. 1–10.
- [34] D. P. Sahoo, P. H. Nguyen, R. S. Chakraborty, and D. Mukhopadhyay, "On the architectural analysis of arbiter delay PUF variants," *Cryptology ePrint Archive*, Report 2016/057, 2016. [Online]. Available: <https://eprint.iacr.org/2016/057>
- [35] F. Ganji, S. Tajik, and J.-P. Seifert, "Why attackers win: On the learnability of XOR arbiter PUFs," in *Proc. Int. Conf. Trust Trustworthy Comput.*, 2015, pp. 22–39.
- [36] J. W. Lee, D. Lim, B. Gassend, G. E. Suh, M. van Dijk, and S. Devadas, "A technique to build a secret key in integrated circuits for identification and authentication applications," in *Proc. Symp. VLSI Circuits*, 2004, pp. 176–179.
- [37] N. K. Shah, D. Chatterjee, B. Sapui, M. Debdeep, and A. Basu, "Introducing recurrence in strong PUFs for enhanced machine learning attack resistance," *IEEE Trans. Emerg. Sel. Topics Circuits Syst.*, vol. 11, no. 2, pp. 319–332, Jun. 2021.
- [38] A. A. Siddhanti, S. Bodapati, A. Chattopadhyay, S. Maitra, D. Roy, and P. Stănică, "Analysis of the strict avalanche criterion in variants of arbiter-based physically unclonable functions," in *Proc. Int. Conf. Cryptol. India*, 2019, pp. 556–577.
- [39] J. Tobisch, A. Aghaie, and G. T. Becker, "Combining optimization objectives: New modeling attacks on strong PUFs," *IACR Trans. Cryptographic Hardware Embedded Syst.*, vol. 2021, pp. 357–389, 2021.
- [40] J. J. M. Chan, P. Thulasiraman, G. Thomas, and R. Thulasiram, "Ensuring quality of random numbers from TRNG: Design and evaluation of post-processing using genetic algorithm," *J. Comput. Commun.*, vol. 4, no. 4, pp. 73–92, 2016.
- [41] Y. Dodis, L. Reyzin, and A. Smith, "Fuzzy extractors: How to generate strong keys from biometrics and other noisy data," in *Proc. Int. Conf. Theory Appl. Cryptographic Techn.*, 2004, pp. 523–540.
- [42] S. S. Roy, C. Rebeiro, and D. Mukhopadhyay, "Theoretical modeling of the Itoh-Tsujii inversion algorithm for enhanced performance on k-LUT based FPGAs," in *Proc. Des. Autom. Test Europe*, 2011, pp. 1–6.
- [43] P. Santikellur, A. Bhattacharyay, and R. S. Chakraborty, "Deep learning based model building attacks on arbiter PUF compositions," *Cryptol. ePrint Arch.*, 2019.
- [44] U. Chatterjee, S. Chatterjee, D. Mukhopadhyay, and R. S. Chakraborty, "Machine learning assisted PUF calibration for trustworthy proof of sensor data in IoT," *ACM Trans. Des. Automat. Electron. Syst.*, vol. 25, no. 4, pp. 1–21, Jul. 2020.



Kuheli Pratihara is currently working toward the PhD degree with the Department of Computer Science and Engineering, IIT Kharagpur under the supervision of Prof. Debdeep Mukhopadhyay and Dr. Rajat Subhra Chakraborty. Her primary research work involves design and lightweight implementation of hardware security primitives like PUFs, TRNGs etc.



Urbi Chatterjee received the PhD degree from the Department of Computer Science and Engineering, IIT Kharagpur, in 2020. She is currently an assistant professor with the Department of Computer Science and Engineering, IIT Kanpur. Her primary research interests include design of lightweight authentication and key exchange protocols based on PUFs for the IoTs.



Rajat Subhra Chakraborty (Senior Member, IEEE) is an associate professor with the Department of Computer Science and Engineering, IIT Kharagpur. His area of research is hardware security, VLSI design, and digital content protection through watermarking. He is a senior member of the ACM.



Manaar Alam is currently working toward the PhD degree with the Department of Computer Science and Engineering, IIT Kharagpur, since 2016. His primary research interests include application of DL in hardware and software security, security evaluation of DL algorithms, and implementations.



Debdeep Mukhopadhyay (Senior Member, IEEE) received the PhD degree from IIT Kharagpur. Currently, he is a full professor with IIT Kharagpur. His research interests include hardware security, micro-architectural attacks, cryptography, VLSI. He is a senior member of the ACM, and the recipient of the prestigious Shanti Swarup Bhatnagar prize 2021.

▷ For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.